DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE

TM-50L

USAREUR-AF

CONCEPTS GUIDE — TM-50L COMPANION

ADVANCED SOFTWARE ENGINEER

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

12 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

DRAFT

CONCEPTS GUIDE — TM-50L COMPANION

ADVANCED SOFTWARE ENGINEER

MAVEN SMART SYSTEM (MSS)

HEADQUARTERS, UNITED STATES ARMY EUROPE AND AFRICA Wiesbaden, Germany 2026

PURPOSE: This guide extends the mental models established in the TM-40F Concepts Guide to advanced software engineering on MSS. Prerequisite: TM-40F Concepts Guide and TM-40F qualification.

DISTRIBUTION RESTRICTION: Approved for public release; distribution is unlimited.

TABLE OF CONTENTS

1. From Engineer to Platform Engineer
2. Platform Architecture Mental Model — Advanced
3. Ontology Design Patterns at Scale
4. OSDK Application Architecture
5. Security Engineering on MSS
6. CI/CD and Automated Promotion — Engineering the Governance Workflow
7. Code Review as a Team Quality Mechanism
8. Engineering Documentation as Operational Infrastructure
9. Advanced Failure Modes — What TM-50L Engineers Get Wrong

DRAFT

SECTION 1 — FROM ENGINEER TO PLATFORM ENGINEER

BLUF

TM-50L is not about writing more complex code than TM-40F. It is about taking ownership of the platform's code health, architectural patterns, and engineering standards — and shaping how the entire team writes, reviews, tests, and deploys software.

The Transition

At TM-40F, a software engineer's primary contribution is delivering working software: transforms that process data correctly, OSDK applications that serve users, integrations that connect systems. The measure of success is whether the code works.

At TM-50L, the measure of success is different. The advanced software engineer asks: does the code work, and does it work in a way that makes the platform stronger over time? Those are different questions. A transform can process data correctly while accumulating technical debt that will collapse under production load in six months. A Workshop application can function for its current users while being architected in a way that cannot be extended to serve the next mission requirement. Code that "works" is necessary but not sufficient at TM-50L.

The TM-50L engineer operates across three dimensions simultaneously:

Dimension	TM-40F Focus	TM-50L Focus
Delivery	Build features that work	Build features that work sustainably
Standards	Follow team standards	Define and enforce team standards
People	Receive code review	Lead code review; develop others

Engineering Leadership Without the Title

Most TM-50L engineers will not hold a leadership position in the traditional Army sense. They will not be the officer in charge or the senior NCO. But they exercise engineering leadership through every interaction with the codebase and the team: the patterns they establish, the code reviews they conduct, the technical decisions they document and defend.

This form of leadership is influence without authority. It requires a different skill set than positional leadership. The TM-50L engineer cannot order a teammate to write better code. They must demonstrate through example what good code looks like, explain through review why a design decision matters, and create systems (checklists, templates, CI gates) that make the right approach the path of least resistance.

Engineering leadership at this level shapes the platform's trajectory. A TM-50L engineer who establishes a clean pattern for error handling in Foundry transforms creates a standard that dozens of future transforms will follow. A TM-50L engineer who lets bad patterns go in code review creates a standard that is equally durable — but in the wrong direction.

The Floor-Raising Imperative

BLUF: The TM-50L engineer's job is to raise the floor on quality across the team, not just to build excellent systems personally.

A team with one excellent engineer and eight mediocre engineers produces mediocre outcomes at scale. The excellent engineer can write excellent code for their own work, but they cannot review all code, fix all designs, or prevent all production incidents alone. The force multiplier for a TM-50L engineer is not their individual output — it is their ability to make every engineer on the team more effective.

The mechanisms for raising the floor: - **Code review**: Not just approving or rejecting, but teaching. Every review comment is an opportunity to explain why a better approach exists. - **Pattern establishment**: When a common problem is solved well, extract the solution into a shared pattern, library, or template. The next engineer who faces the same problem starts from a better place. - **Documentation**: Architecture decisions, gotchas, and lessons from production incidents become institutional knowledge instead of tribal knowledge held by one person. - **CI/CD gates**: Automate quality enforcement wherever possible. A linter that catches a naming convention violation is better than a human reviewer who might miss it.

Vignette — The New Transform Pattern

A TM-50L SWE on the USAREUR-AF MSS platform team observes that four different engineers have independently implemented error handling in Foundry transforms in four different ways. None of the approaches are wrong, but two lack adequate logging for debugging production failures, one suppresses exceptions silently, and the fourth is verbose but difficult to read.

The TM-40F response would be to fix the problem in your own transforms and move on. The TM-50L response is different: document the four approaches, explain the tradeoffs, define a team standard for error handling, implement it as a shared utility or code template, and update the code review checklist to verify the standard on new transforms. The one-time investment in standardization prevents the problem from recurring across the next fifty transforms the team writes.

DRAFT

SECTION 2 — PLATFORM ARCHITECTURE MENTAL MODEL — ADVANCED

BLUF

TM-50L engineers understand not just how to write transforms but how the Foundry platform executes them: Spark execution, dataset partitioning, compute resource allocation, and performance reasoning at scale. The diagnostic skill — moving from symptoms to root cause — is the advanced competency.

The Spark Execution Model

Foundry's data transformation layer is built on Apache Spark. Understanding how Spark executes code is essential for reasoning about performance at scale — and for diagnosing failures when they occur.

At a conceptual level, Spark processes data in two phases:

Transformation phase (lazy): When code calls `.filter()`, `.join()`, or `.select()`, Spark does not execute immediately. It builds a directed acyclic graph (DAG) of operations — a logical execution plan. This plan is optimized before any data is read. The implication for TM-50L engineers: the order in which transformations are written affects the execution plan, and the optimizer has limits. Push filters as early as possible in the chain. Avoid wide operations (shuffles, joins) until the dataset has been reduced.

Action phase (eager): When code calls `.write()` or `.count()` — actions that produce a result — Spark materializes the plan and executes it. This is when actual computation happens, distributed across worker nodes.

The key architectural insight: Spark moves data across the network during shuffle operations. A join between two large datasets requires both datasets to be reorganized by join key across the cluster — this is expensive. Every wide operation is a potential performance bottleneck. The TM-50L engineer designs transforms to minimize shuffles: reduce data early, filter before joining, partition by the key that downstream operations will use.

Dataset Partitioning

Foundry datasets are stored as partitioned files (typically Parquet). Partitioning determines how data is physically organized on disk and how Spark reads it.

Good partitioning: Data is divided into roughly equal-sized partitions along a dimension that matches the most common query pattern. For a dataset of SITREP records partitioned by unit and date, a query for all records from a single brigade on a specific date reads a small number of partitions and touches a small fraction of the total data.

Bad partitioning: Data is divided unevenly (partition skew) or along a dimension unrelated to common query patterns (forcing full scans). The pathological case is a dataset with millions of records in one partition and dozens in every other. The single large partition becomes a bottleneck: one worker node processes most of the work while the rest sit idle.

Partitioning design decisions at TM-50L level:

Scenario	Design Choice	Reasoning
High-frequency queries by unit + date	Partition by unit_id, date	Prunes irrelevant partitions on every query
Large, rarely filtered dataset used for joins	Partition by join key	Reduces shuffle size in downstream joins
Small reference dataset (<1M rows)	Single partition acceptable	Partitioning overhead not justified
Dataset with extreme cardinality skew	Bucket by hashed key	Distributes skewed data evenly

Compute Resource Allocation

Foundry allocates compute resources (CPU cores, memory) to transform builds. TM-50L engineers understand when and how to configure resource allocation — and, more importantly, when resource configuration is the wrong solution to a performance problem.

The common mistake: a transform fails with an out-of-memory (OOM) error, and the engineer's first response is to request more memory. This sometimes works. More often, OOM errors indicate a design problem that more memory will not permanently fix — it will only push the failure to a larger dataset.

Diagnostic reasoning for OOM errors:

1. What is the dataset size? Is it growing? If the dataset doubled in size, would the transform fail again?
2. Where in the execution plan does memory spike? Broadcasting a large reference dataset, collecting a large result to the driver, or operating on an unskewed join are common culprits.
3. Can the operation be restructured to avoid materializing large intermediate datasets? Streaming aggregations, incremental processing, and pushdown filters often eliminate OOM errors without touching resource configuration.

Diagnostic reasoning for timeout errors:

1. Is the timeout hitting because the transform is doing too much work, or because a single step is slow (Amdahl's Law — a slow serial step limits parallelism)?
2. Is the slow step a shuffle? What is the shuffle size? Can it be reduced?
3. Is the transform reprocessing data it has already processed? Incremental processing with watermarks (TM-50L task territory) eliminates reprocessing of historical data.

Symptoms-to-Root-Cause Reasoning

The following table maps common production symptoms to diagnostic hypotheses. TM-50L engineers use this framework before changing resource configuration or requesting platform support.

Symptom	First Hypothesis	Diagnostic Step
OOM on worker node	Partition skew or large broadcast join	Check partition size distribution; check for `broadcast()` on large datasets
OOM on driver node	`collect()` or `toPandas()` on large dataset	Review code for driver-side data collection
Timeout on shuffle-heavy join	Excessive shuffle size	Profile shuffle read/write bytes; evaluate join order and filter placement
Transform slow at 10M rows, fails at 100M rows	Non-linear scaling due to cartesian product or unintended full scan	Review join conditions; check for missing join key filters
Correct output at small scale, wrong output at large scale	Non-deterministic operation or race condition in distributed execution	Review aggregation logic for operations that are not associative/commutative
Build succeeds but downstream dataset is empty	Predicate pushdown filtered all data	Verify filter conditions against actual data distributions

Vignette — Diagnosing a Theater-Scale Transform Failure

The USAREUR-AF logistics integration team submits a new transform that joins unit equipment readiness records against a theater-wide parts-availability dataset. At development scale (10K records), it runs in under two minutes. In the production environment (40M equipment records × 800K parts records), it fails with an OOM error on the shuffle step.

The TM-50L SWE conducting the review does not recommend increasing the memory configuration. Instead, they trace the execution plan: the transform performs a join without a preceding filter on the equipment dataset, resulting in a full 40M × 800K shuffle. The fix is to filter equipment records to the relevant AOR and date range before the join, reducing the input to the shuffle by roughly 95%. The revised transform passes at production scale without any change to resource configuration. The root cause was design, not resources.

DRAFT

SECTION 3 — ONTOLOGY DESIGN PATTERNS AT SCALE

BLUF

At TM-50L level, the SWE contributes to Ontology design decisions that affect the entire platform. The critical competency is evaluating a design proposal for scalability before it is implemented — not after it fails under production load.

Why Ontology Design Is an Engineering Concern

The Ontology is not a configuration exercise — it is a software architecture decision. Object Types, Properties, Links, and Actions define the data model that Workshop applications, OSDK integrations, and analytical pipelines consume. A design decision made in the Ontology today becomes a constraint on everything built on top of it. Changing Ontology design retroactively is expensive: applications break, integrations require updates, and data must be migrated.

The TM-40F engineer learns to implement Ontology definitions correctly. The TM-50L engineer learns to evaluate Ontology definitions for long-term scalability before they are merged.

Object Type Inheritance Patterns

Foundry's Ontology does not support classical object inheritance in the object-oriented sense. There is no "extends" relationship between Object Types. However, TM-50L engineers can model inheritance-like relationships through two patterns, each with distinct tradeoffs:

Pattern 1: Single Object Type with a type discriminator property. A single Object Type (e.g., `Personnel`) includes a `role` property that distinguishes subtypes (e.g., "ORSA", "Software Engineer", "Knowledge Manager"). All properties for all subtypes exist on every object; properties irrelevant to a given subtype are null.

Tradeoff: Simpler Ontology structure, fewer Link Types, easier querying across the full population. Cost: property sparsity for objects where most properties are null, which wastes storage and creates schema confusion for consumers who do not understand which properties apply to which subtype.

Pattern 2: Separate Object Types per subtype with a shared Link to a parent type. An `ORSAPersonnel` Object Type and a `SoftwareEngineer` Object Type each link to a `Personnel` parent Object Type. Type-specific properties live on the subtype Object Type; shared properties live on the parent.

Tradeoff: Clean separation of concerns, no property sparsity, clearer API contract per subtype. Cost: complex Ontology graph, multi-hop traversal required for queries that span parent and child, higher maintenance burden when adding new subtypes.

Design guidance for TM-50L engineers: Use the discriminator pattern when subtypes share most properties and differ only in a few fields. Use the separate-type pattern when subtypes have fundamentally different property sets and are accessed primarily within their own type context. Never use the separate-type pattern for fine-grained distinctions that will produce many Object Types (>10) with overlapping semantics — this creates an unmaintainable Ontology.

Sparse vs. Dense Property Design

Dense design: All properties are expected to have values for all objects. A `Unit` Object Type with `name`, `UIC`, `parent_UIC`, `echelon`, `AOR`, `commanding_officer` is dense — these properties apply to every unit in the theater.

Sparse design: Most properties are null for most objects. A `PersonnelRecord` Object Type with 80 properties, of which any given person has 15-20 populated, is sparse. Sparse design is a warning sign.

Sparse Ontology designs often indicate that the designer conflated multiple distinct Object Types into one, or that optional metadata was added as top-level properties instead of being modeled as linked objects or stored in a secondary dataset. The TM-50L engineer identifies sparse designs during review and challenges them: what problem does this structure solve, and is there a denser model that solves the same problem more cleanly?

The operational cost of sparse designs: Workshop widgets display null values for most properties, OSDK queries return objects with mostly-empty property sets, and computed properties over sparse properties produce misleading aggregates.

Computed Property Performance Tradeoffs

Computed properties — properties whose values are calculated from other properties or from linked objects — are powerful and expensive. Every request that touches a computed property requires the platform to evaluate the computation at query time.

Performance considerations for TM-50L engineers:

Computed Property Type	Performance Profile	When to Use
Simple arithmetic on the same object's properties	Low cost, evaluates locally	Always acceptable
Aggregation over directly linked objects (1-hop)	Moderate cost, scales with link count	Acceptable for infrequent access, beware objects with thousands of links
Aggregation over 2+ hop traversals	High cost, potentially exponential with link counts	Avoid; materialize as a dataset property instead
Computation requiring external data lookup	Variable; can become a platform-wide bottleneck	Avoid in high-frequency-access Object Types

The TM-50L engineer's rule: if a computed property will be accessed by Workshop tiles that serve many users simultaneously, model the computation carefully. A property that is cheap to compute for one user becomes expensive when ten Workshop instances request it simultaneously for large Object Sets.

Large-Scale Link Traversal Performance

Link traversal — following relationships between Object Types to retrieve related objects — is one of the most expensive operations in the Ontology. At scale, link traversal performance determines whether an application feels responsive or broken.

The key variable: **fan-out**. A link from `Brigade` to `Unit` has low fan-out (brigades contain a bounded number of units). A link from `Personnel` to `TrainingRecord` may have high fan-out if each person has hundreds of records. A query that traverses a high-fan-out link for thousands of objects simultaneously can produce millions of individual link resolutions — and application latency measured in tens of seconds.

Design interventions for high-fan-out scenarios:

1. **Limit traversal depth at the application layer.** OSDK queries should specify the traversal depth explicitly and avoid open-ended traversals.
2. **Materialize aggregate values.** If an application needs to display "number of training records for each person," compute that count as a dataset property and expose it directly rather than traversing the link at query time.
3. **Paginate large linked sets.** Never load all linked objects for a large Object Set in a single query. Page results and load incrementally.
4. **Evaluate link direction.** Links are bidirectional in the Ontology, but traversal cost may differ by direction. Profile both directions for high-fan-out links.

Pre-Implementation Review Framework

Before approving an Ontology design proposal, the TM-50L engineer should evaluate it against these questions:

1. **Scale projection:** How many objects will this Object Type contain at initial deployment? In 12 months? In 36 months? Does the design hold at projected scale?
2. **Access pattern fit:** What are the primary query patterns for this data? Does the property and link structure support those patterns efficiently?
3. **Property sparsity:** What percentage of properties are expected to be null for the average object? If >30%, challenge the design.
4. **Computed property cost:** For every computed property, estimate the cost at peak load. Could it be materialized instead?
5. **Link fan-out:** For every Link Type, what is the expected maximum cardinality on each side? Flag any links where one side can have >1,000 related objects.

6. **Change impact:** If this design is wrong and needs to change in six months, what will break and how much will it cost to fix?

SECTION 4 — OSDK APPLICATION ARCHITECTURE

BLUF

At TM-50L level, OSDK applications go beyond displaying Object data. Advanced patterns include caching strategies, real-time updates, multi-Object-Type design, and authentication/authorization for applications serving multiple roles with different access levels.

The Architecture Decision That Precedes Everything

Before designing any advanced pattern into an OSDK application, answer one question: what is the access pattern? How many users will use this application simultaneously? How frequently will they query data? How much data will each query touch? How stale can the data be before it is operationally unacceptable?

These answers determine every architectural decision that follows. An OSDK application serving 5 analysts querying daily historical data has different requirements from one serving 200 planners querying near-real-time force disposition data during an exercise. Building the second pattern for the first use case wastes platform resources and engineering time. Building the first pattern for the second use case produces an application that fails operationally when it matters most.

Caching Strategies for Frequently Accessed Object Sets

The Ontology is not a high-frequency transactional database. Applications that query the same Object Sets repeatedly without caching drive unnecessary platform load and produce inconsistent user experience as query latency varies.

Client-side caching in OSDK applications:

The OSDK client maintains an in-memory cache of Objects it has loaded. Subsequent accesses to the same Objects return cached values without network round-trips. The TM-50L engineer understands the cache invalidation model: when does the cache update? What is the staleness window? For near-real-time applications, staleness of a few minutes may be unacceptable. For historical analysis applications, staleness of hours is irrelevant.

Application-layer caching:

For Object Sets that are queried frequently by many users (e.g., a current force disposition Object Set that every planner queries at the start of each session), the application layer should cache the result and serve subsequent users from cache rather than querying the Ontology for each user. Cache invalidation trigger: when the underlying data changes, invalidate the cache and reload.

Cache invalidation failure mode: Caching that never invalidates is indistinguishable from stale data. Operationally, an application that shows yesterday's force positions because its cache never refreshed is actively harmful. Define cache TTL explicitly and document it in the application's operational runbook.

Real-Time Updates via Ontology Subscriptions

OSDK supports subscriptions — the application registers interest in an Object Set and receives updates when Objects in that set change. This enables near-real-time applications without polling.

Architecture considerations:

- Subscriptions generate network traffic proportional to the rate of underlying data change. For a slowly-changing Object Set (unit equipment status updated daily), subscriptions are low overhead. For a rapidly-changing Object Set (event stream ingested every 30 seconds), subscriptions may generate more traffic than polling.
- Applications that maintain subscriptions across many Object Types simultaneously may hit platform subscription limits. Design applications to subscribe only to the data they are actively displaying, and unsubscribe when the user navigates away.
- Subscription failures must be handled gracefully. An application that silently stops receiving updates is worse than one that displays an explicit "data may be stale" indicator.

Multi-Object-Type Application Design

Complex operational applications typically display data from multiple Object Types simultaneously: personnel, equipment, units, events, and their relationships. Designing these applications well requires explicit decisions about data loading strategy.

Load-what-you-need principle: Do not load all data from all Object Types at application startup. Load the minimum set needed to render the initial view, then load additional data as users navigate or filter. An application that loads 50,000 Objects across five types at startup will have a long initial load time and poor perceived performance, even if subsequent interactions are fast.

Relationship navigation: When a user selects an object and the application needs to display related objects from a linked Type, load those related objects on demand — not preemptively. Most users will not navigate to most relationships. Preemptive loading wastes bandwidth and platform resources.

Consistent identity across Object Types: In applications that display the same real-world entity through multiple Object Types (e.g., a unit displayed through both its `Unit` Object Type and its `UnitEquipment` Object Type), the application must maintain consistent identity mapping. A user

selecting a unit in one panel should filter all related panels to that unit automatically. Implement this through application state management, not through separate queries.

Authentication and Authorization for Multi-Role Applications

OSDK applications that serve multiple roles with different access levels require explicit authorization architecture. The Ontology's access controls (CBAC, Object Type-level permissions) are the enforcement layer, but the application's design must align with the permission model.

Design principle: surfaces, not data. The application should show users only the data surfaces they are authorized to access. A planner and a commander using the same application may see different Object Types, different property sets on the same Object Type, or different Actions available on the same object. These differences are not cosmetic — they are authorization requirements.

Implementation pattern: Query the current user's role from the authentication context at application load. Use the role to determine which application sections to render, which Object Types to query, and which Actions to expose. Do not rely solely on the Ontology's enforcement layer to hide unauthorized data — the application should not request data it has no authorization to display. Defense in depth: the Ontology enforces; the application cooperates.

Authorization failure mode: An application that queries unauthorized data and simply renders nothing when the Ontology returns empty results looks broken to the user. Users will not understand that they received empty results because they lack access; they will report the application as broken. Design explicit access-level awareness into the application: if a user is not authorized to see a section, render a clear indicator ("Access to this section requires [role]") rather than an empty panel.

SECTION 5 — SECURITY ENGINEERING ON MSS

BLUF

Security is a design constraint applied from the beginning of every system, not a feature added at the end. At TM-50L level, the engineer evaluates every component for its security model before writing the first line of code.

The Security Design Mindset

TM-40F engineers learn to implement security controls: apply CBAC markings, use HTTPS, rotate credentials. TM-50L engineers design security in from the architecture phase: before choosing a data storage pattern, an integration approach, or an application architecture, ask — what is the security model of this design, and does it satisfy the requirement?

The questions that precede every design decision at TM-50L level:

1. **Authentication:** Who is allowed to access this system component? How is their identity verified? What happens when authentication fails?
2. **Authorization:** Of authenticated users, who is allowed to perform which operations? How are permissions granted, revoked, and audited?
3. **Data markings:** What is the sensitivity classification of the data this component handles? Are markings enforced at ingestion, storage, and access? Can markings be accidentally stripped?
4. **Audit logging:** What operations need to be audited? Are audit logs tamper-resistant? Who can access them?
5. **Least privilege:** Does each system component have only the permissions required for its function? A transform that reads two datasets and writes one should not have platform-wide write access.

Column-Level Permissions in Datasets

Foundry supports column-level access controls on datasets — specific columns can be restricted to users with specific roles, while the remainder of the dataset is broadly accessible. This pattern enables a single dataset to serve multiple audiences with different access levels.

When column-level permissions are appropriate: A personnel dataset that contains both administrative fields (name, unit, position — broadly accessible) and sensitive fields (disciplinary records, medical flags — restricted). Column-level permissions allow the same dataset to serve both the administrative office and the personnel officer without duplicating the data.

Security engineering considerations: - Column-level permissions must be documented. A consumer who queries the dataset and receives null values for restricted columns may not know why. Document which columns are restricted and under what access conditions. - Column restrictions do not prevent a consumer from knowing that restricted columns exist. If the existence of a column is itself sensitive, evaluate whether a separate dataset is the correct design. - Joins between datasets with column-level restrictions can leak information indirectly. If restricted column A is the join key used to produce dataset B, dataset B now implicitly encodes information from column A. Evaluate derived datasets for indirect information exposure.

Object Type-Level Access Controls in the Ontology

The Ontology's CBAC model controls which users and groups can read, write, or execute Actions on specific Object Types. TM-50L engineers design Ontology access control schemas, not just implement them.

Access control design principles:

- Grant access to roles, not individuals. Individual access grants create a maintenance burden and break when personnel rotate. Role-based grants are inherited by anyone holding the role.

- Apply the principle of minimum necessary access. An analyst who needs to read equipment status Objects does not need write access. An operator who needs to execute readiness update Actions does not need access to personnel Objects.
- Restrict Action execution separately from Object read access. A user can be authorized to view an Object's properties while being unauthorized to execute Actions that modify it. These are independent permissions.
- Review access control schemas when personnel rotate into or out of roles. Access that was appropriate for one incumbent may not be appropriate for all successors.

Action Authorization in Workshop Applications

Workshop Actions are the primary mechanism by which users modify Ontology data from applications. Each Action must have an explicit authorization model.

The TM-50L security review checklist for Actions:

Check	Question	Failure Mode
Actor authorization	Is the user executing this Action verified as authorized at the Ontology layer, not just the UI layer?	UI hides the button but the Action remains callable via API
Input validation	Are all Action inputs validated before execution?	Malformed input causes data corruption or exposes system behavior
Rollback capability	Can the effect of this Action be reversed?	Accidental or malicious Action execution with no recovery path
Audit trail	Is Action execution logged with actor identity, timestamp, and parameters?	No forensic record of who changed what and when
Scope limitation	Does the Action modify the minimum set of Objects required?	Over-scoped Action modifies objects outside the intended set

What a Security Review of a Code PR Looks Like at TM-50L

Security review is not a separate audit performed by a security team — it is integrated into the code review that TM-50L engineers conduct on every PR. The following checks are mandatory at TM-50L level:

Credential handling: Are credentials stored in environment variables or the Foundry credential store? Any hardcoded credential — even a test credential, even a credential for a low-privilege account — is a finding that blocks the PR.

Data flow tracing: Follow the data from ingestion to output. At each step, does the data carry its markings? Is there any step where markings could be stripped or downgraded? Is there any step where restricted data could be written to an unrestricted location?

External connectivity: Does this code open a connection to any external system? If so, is the external system on the approved integration list? Is the connection authenticated and encrypted? Is the data being sent to the external system reviewed for sensitivity?

Logging review: What does this code log? Are log statements inadvertently capturing PII, credentials, or sensitive data payloads? Logging sanitized identifiers (record IDs, hashed values) is acceptable; logging field values from sensitive datasets is not.

Error handling review: Do error conditions expose system internals (stack traces, schema information, connection strings) in error messages returned to users? Error messages should describe the failure category without exposing implementation details.

SECTION 6 — CI/CD AND AUTOMATED PROMOTION — ENGINEERING THE GOVERNANCE WORKFLOW

BLUF

The TM-50L engineer designs and maintains the automated pipeline that moves code from development to production. An effective CI/CD pipeline enforces governance requirements without creating bottlenecks that slow the team.

What CI/CD Enforces on MSS

The MSS platform team's CI/CD pipeline enforces three categories of requirements:

Code quality: Automated tests pass, code meets coverage thresholds, static analysis identifies common errors, naming conventions comply with USAREUR-AF governance standards. These checks run on every PR and block merge on failure.

Security requirements: No hardcoded credentials (secret scanning), no connections to unapproved external endpoints, dependency vulnerability scan passes. These checks run on every PR and are non-negotiable blockers.

Documentation requirements: Architecture decision records (ADRs) exist for significant design changes, runbook updates are included for changes that affect operational procedures, and CHANGELOG entries document user-facing changes. These checks run on every PR; missing documentation generates a warning or blocks merge depending on the change classification.

Pipeline Design Principles

Fast feedback: The PR author should know within minutes whether their changes pass quality gates. A CI pipeline that takes 45 minutes to run discourages frequent commits and leads engineers to batch changes together — which makes review harder, increases the blast radius of failures, and defeats the purpose of CI. Invest in parallelizing tests and running fast checks first (lint before full test suite).

Fail fast, fail clearly: When a CI check fails, the failure message should tell the engineer exactly what failed, why it failed, and how to fix it. A generic "build failed" with a link to a log file is not useful. Structured failure messages that identify the specific check, the specific file, and a reference to the relevant standard are actionable.

Idempotent checks: CI checks should produce the same result when run multiple times on the same code. Checks that depend on external service availability, time-of-day, or random seeds produce flaky results that erode trust in the pipeline. Engineers who learn to ignore flaky CI failures will also ignore real failures.

Gate progression: Not all checks need to run before merge. Classify checks by severity:

Severity	Action	Examples
Hard block	Merge prevented until resolved	Test failure, credential scan failure, security finding
Soft block	Warning logged; requires explicit override with justification	Coverage below threshold, missing documentation
Advisory	Informational only	Style suggestions, optional improvements

The Governance Bottleneck Problem

Governance requirements — naming conventions, documentation requirements, testing thresholds — are operationally necessary. They are also a common source of bottlenecks. When governance checks are manual (requiring a human review step), they create a queue. When the queue backs up, engineers wait. When engineers wait, they find workarounds.

The TM-50L engineer's approach to this problem: automate everything that can be automated, and design the automated checks to be informative rather than opaque. A naming convention check that blocks a PR and tells the engineer "Dataset name 'unit_data_v2_final' violates USAREUR-AF naming standard — expected format: [domain]*[entity]*[version] (e.g., log_unit_readiness_v1)" enables the engineer to fix the problem immediately without a human reviewer explaining it. The automated check becomes a teaching tool.

Manual governance review should be reserved for decisions that require human judgment: significant architecture changes, new external integrations, changes to access control schemas. These warrant a human reviewer. Naming convention compliance does not.

Promotion Workflow Design

Foundry's promotion workflow moves a verified code branch from a development environment to a production environment. The TM-50L engineer designs this workflow to include checkpoints that enforce the team's deployment standards.

Recommended promotion gate sequence: 1. All CI checks pass (automated) 2. PR approved by one TM-50L-qualified reviewer (human) 3. Staging environment deployment and smoke test (automated) 4. C2DAO sign-off for changes affecting production Ontology or CBAC policies (human) 5. Production deployment with automated rollback on health check failure (automated)

The key design principle: automate the verification steps, and reserve human review for the approval decisions. Humans are slow and expensive at verification. They are not expensive at approval if the verification has already been done automatically.

SECTION 7 — CODE REVIEW AS A TEAM QUALITY MECHANISM

BLUF

TM-50L engineers lead code reviews. Effective code review evaluates correctness, design, maintainability, security, and operational risk — and delivers feedback in a way that improves the reviewer without demoralizing them.

The Four Dimensions of Code Review

Correctness: Does the code do what it is supposed to do? This is the dimension most engineers focus on. It is necessary but not sufficient. A transform that produces the correct output today can fail when the input schema changes, when the dataset grows by 10x, or when a dependency is updated. Correctness review includes: Does it handle edge cases? What happens when the input is empty? When a join produces no matches? When an API call times out?

Design: Is this the right approach to the problem? A correct solution can be the wrong design. Over-complex solutions introduce maintenance burden without adding value. Under-designed solutions handle today's requirement but cannot be extended to tomorrow's. Design review asks: Is there a simpler way to solve this? Will this design hold as requirements evolve? Are the boundaries of this component's responsibility clearly defined and correct?

Maintainability: Will someone else understand this code in six months? The "someone else" might be the original author, who will not remember the details of what they wrote. Maintainability review asks: Are variable and function names clear? Is complex logic commented? Does the structure of the code reflect

its purpose? If this code needs to be debugged at 0300, can a qualified engineer understand what it is doing quickly?

Security and operational risk: Does this change introduce a vulnerability? What is the blast radius if it fails in production? Security review is covered in Section 5. Operational risk review asks: What is the worst-case failure mode for this change? Does it affect one user, one unit, or the entire theater? Is the change reversible? Are there monitoring and alerting hooks so that failures are detected before they cause operational impact?

Blast Radius Analysis

Every production change has a blast radius — the scope of impact if the change fails. TM-50L engineers make blast radius assessment explicit in code review.

Blast Radius	Description	Example
Local	Affects one user or one non-critical pipeline	A personal dashboard transform fails
Unit	Affects one unit's data or workflows	A brigade readiness rollup transform produces wrong outputs
Platform	Affects a shared service or the Ontology schema	An OSDK change breaks a widely-used Object Type's API contract
Theater	Affects all USAREUR-AF users or mission-critical pipelines	A CBAC policy change removes access for all G3 users

Changes with theater-level blast radius require additional review steps, coordination with C2DAO, and a documented rollback plan before deployment. Code review should identify the blast radius of a change and ensure that the review process matches the risk.

Giving Effective Code Review Feedback

Code review feedback that is vague, harsh, or condescending creates defensiveness and slows the team's learning. Code review feedback that is specific, respectful, and educational accelerates the reviewer's development and raises team quality over time.

Principles for TM-50L code review feedback:

1. **Separate the code from the person.** "This function is doing three unrelated things" is about the code. "You wrote a confusing function" is about the person. Review the code.
2. **Explain the why, not just the what.** "Change this to X" leaves the reviewer guessing why X is better. "Change this to X because it handles the empty-input case that Y silently drops" teaches the reviewer something they will carry forward.

3. **Distinguish blocking feedback from suggestions.** Label feedback explicitly: "Blocking: this creates a security vulnerability" versus "Suggestion: this could be simplified." Reviewers should not have to guess which feedback requires a change before merge.
4. **Acknowledge good work.** Code review should not be exclusively a list of problems. When a reviewer implemented a complex pattern correctly, or handled an edge case you would not have thought of, say so. This builds the relationship and establishes the standard for what "good" looks like.
5. **Calibrate to the reviewer's level.** A TM-30-level engineer submitting their first complex transform needs different feedback than a TM-40F engineer who has made an architectural misjudgment. Adjust the depth and style of feedback to where the reviewer is in their development.

The Reviewer Development Obligation

TM-50L engineers have an obligation to develop the engineers whose code they review. Every code review is a mentorship opportunity. The cumulative effect of many well-executed code reviews is a team whose quality improves over time.

The reviewer development mindset in practice: when a reviewer makes a design mistake you have seen before, do not just correct it in the PR — take five minutes after the review to discuss why the pattern matters. When a reviewer writes code that reflects a misunderstanding of how the platform works, point them to the relevant section of TM-40F or TM-50L. When a reviewer does something well that you want to reinforce, be explicit about it.

SECTION 8 — ENGINEERING DOCUMENTATION AS OPERATIONAL INFRASTRUCTURE

BLUF

At TM-50L level, documentation is an engineering artifact with the same lifecycle as code. Missing documentation is a defect. The operational cost becomes apparent when a system fails at 0300 and no one knows how to recover it.

What Must Be Documented

Architecture Decision Records (ADRs): Every significant design decision should be documented as an ADR: the context, the options considered, the decision made, and the rationale. ADRs serve two purposes. First, they give future engineers the reasoning behind a design so they do not accidentally undo it without understanding why it exists. Second, they serve as a forcing function: engineers who know they must document their reasoning think more carefully about their decisions.

ADR format (adapted for MSS):

```
ADR-[number]: [Short title]
Date: [YYYY-MM-DD]
Status: [Proposed / Accepted / Superseded]
Context: [What problem does this decision address?]
Decision: [What did we decide to do?]
Rationale: [Why did we choose this option over alternatives?]
Consequences: [What are the tradeoffs of this decision?]
```

API contracts: Every OSDK API, webhook endpoint, and integration contract must be documented. The documentation must specify: input parameters, output schema, authentication requirements, error responses, and rate limits or throughput constraints. The documentation must be versioned — when the API changes, the documentation changes. Consumers of the API must be notified of changes that break backward compatibility.

Data schemas: Every production dataset that other pipelines or applications depend on must have a documented schema. The schema documentation should include: field names, types, descriptions, acceptable value ranges, and known data quality issues. Co-locate schema documentation with the transform that produces the dataset so it is updated when the transform is updated.

Deployment runbooks: Every system component that requires operational maintenance (restart procedures, scaling procedures, credential rotation, dependency update procedures) must have a runbook. The runbook must be written for an engineer who is competent but unfamiliar with this specific system — not for the engineer who built it. Test runbooks by having a second engineer follow them without assistance. If they cannot, the runbook is incomplete.

Incident postmortems: Every production incident above a threshold severity should produce a postmortem document: what happened, what the timeline was, what the root cause was, and what changes are being made to prevent recurrence. Postmortems are not blame documents — they are engineering artifacts. Filed in the team's knowledge repository, they become the institutional memory that prevents the same failure from recurring when a new team rotates in.

Making Documentation Sustainable

Documentation that lives in a separate wiki detached from the code it describes will rot. Developers update the code; the wiki is forgotten. Within six months, the wiki is misleading — worse than no documentation because readers may trust it.

Co-location principle: Documentation should live as close to the code it describes as possible. ADRs in the repository directory they pertain to. Schema documentation in or adjacent to the transform that produces the dataset. API documentation in the API codebase itself, generated from code annotations where possible.

Documentation as a PR requirement: Documentation updates must be required as part of the merge criteria for PRs that change system behavior. The CI check framework described in Section 6 can enforce this: if a PR modifies a transform that produces a documented dataset, a check verifies that the schema documentation file was also updated.

Review cycle: Documentation should be reviewed on a defined cycle, separate from code changes. Quarterly documentation review: pull up the runbook for each critical system and verify it against current reality. Flag outdated sections. Assign updates as work items with the same priority as code maintenance.

The 0300 Test

When designing documentation, apply the 0300 test: if a production system fails at 0300 and a qualified but unfamiliar engineer needs to recover it, does the documentation exist to enable that recovery without waking up the system's original author?

If the answer is no, the documentation is incomplete. This is not a quality-of-life issue — it is an operational readiness issue. On an operational data platform supporting theater-level decision-making, a data pipeline that cannot be recovered without calling one specific engineer who is on leave is a single point of failure.

The documentation that makes the 0300 test possible: - Runbook for each critical pipeline: what healthy looks like, how to diagnose common failure modes, how to restart or roll back - Dependency map: what does this system depend on, and what depends on it? - Contact escalation: if the runbook does not resolve the issue, who is the next call? What information do they need to have ready before calling?

SECTION 9 — ADVANCED FAILURE MODES — WHAT TM-50L ENGINEERS GET WRONG

BLUF

The failure modes at TM-50L level are different from those at TM-40F. They are not beginner mistakes — they are predictable errors made by capable engineers who have access to powerful tools and face genuine operational pressures.

Failure Mode 1 — Over-Engineering

The MSS platform provides sophisticated tools: incremental processing, Ontology subscriptions, complex OSDK patterns, multi-hop link traversals, and streaming integrations. TM-50L engineers have mastered these tools. The failure mode is applying them when simpler approaches would serve the mission better.

Over-engineering has several causes: intellectual preference for elegant solutions over simple ones, defensive engineering ("what if we need this later?"), and status signaling ("complex code demonstrates skill"). All three are traps.

The operational cost of over-engineering: systems that are harder to understand, harder to maintain, harder to debug in production, and harder to hand off when the team rotates. A transform that uses advanced incremental processing with a custom watermark strategy when a full rebuild once daily would suffice is not a better system — it is a more complex one. Complexity has a cost. That cost is paid by every engineer who touches the system after it is built.

The corrective discipline: before adding complexity, ask what simpler approach was rejected and why. If the simpler approach has a concrete operational deficiency (too slow, too resource-intensive, too stale), the complexity is justified. If the simpler approach would have worked, refactor.

Failure Mode 2 — Underinvesting in Observability

Production systems fail in ways that development environments do not reveal. Without observability infrastructure — logging, alerting, metrics — production failures are discovered by users reporting them, not by the engineering team detecting them.

TM-50L engineers who underinvest in observability typically do so because observability instrumentation is not immediately functional: it does not make the system work better today. The investment pays off only when something goes wrong. Until it goes wrong, the investment looks like overhead.

The corrective discipline: treat observability requirements as functional requirements, not optional enhancements. Before a system goes to production, define: what does healthy look like (metrics), how will we know something is wrong (alerting thresholds), and how will we diagnose what went wrong (logging structure). If these cannot be specified, the system is not ready for production.

Minimum observability requirements for TM-50L-level systems: - Transform build success/failure alerting - Data freshness monitoring (last successful run, records processed) - Output quality checks (row counts, null rates on key fields) - OSDK application error rate monitoring - Action execution failure alerting

Failure Mode 3 — Technical Debt Accumulation

Systems that "work" but are built on a fragile foundation accumulate technical debt silently until the debt becomes an operational liability. The failure point is usually a growth event: the dataset doubles in size, a new use case requires schema extension, or the team tries to build a new feature on top of the existing system and finds it cannot be extended without a rewrite.

TM-50L engineers are not immune to this failure. They may build technically sophisticated systems that carry structural debt: documentation gaps, hardcoded assumptions that were correct at build time, performance characteristics that hold at current scale but will degrade, or test coverage that exercises happy paths but not failure modes.

The corrective discipline: treat technical debt as an operational risk, not a future engineering problem. During each sprint, budget time for debt reduction. When a new feature is built on top of an existing system, assess the system's debt before adding to it. Code review should explicitly identify debt created by a PR — if a PR adds a feature in a way that takes on debt, the review should note it and the team should decide whether to accept the debt or address it before merge.

Failure Mode 4 — Security as a Compliance Checkbox

Security reviews, ATO processes, and governance checklists can create a compliance mindset: the goal is to pass the checklist, not to build a secure system. Engineers who adopt this mindset look for the minimum necessary evidence to satisfy each checklist item, not for actual security assurance.

The compliance mindset produces systems that pass reviews and fail in the real world. A credential stored in an approved vault, never rotated, with overly broad permissions, shared across multiple systems, meets the checklist requirement for "no hardcoded credentials" — and is still a security liability.

The corrective discipline is the mindset shift described in Section 5: security is a design constraint, not a compliance requirement. Before building any system component, think through its security model from first principles. The checklist is a verification tool for work that was already designed with security in mind — not a substitute for that design thinking.

Failure Mode 5 — Failing to Develop the Next Generation

TM-50L engineers with heavy operational workloads may deprioritize mentorship and code review quality because these activities feel less urgent than delivering the next system. This is a compounding failure: the team's overall capability stagnates, the TM-50L engineer becomes a bottleneck for quality, and when they rotate out, the platform loses institutional knowledge that was never transferred.

At USAREUR-AF, operational data team personnel rotate. The team that built a system will not be the team that maintains it in two years. If the TM-50L engineer has not invested in developing TM-30 and TM-40F engineers to carry the platform forward, the rotation creates a capability gap that degrades operational readiness.

The corrective discipline: treat engineer development as a mission requirement, not optional. Code review is a teaching opportunity — invest in it accordingly. When a TM-30 or TM-40F engineer is struggling with a concept, take the time to explain it rather than fixing it yourself. Create documentation and patterns that make institutional knowledge portable. The TM-50L engineer's measure of success is not the quality of the systems they build alone — it is the quality of the systems the team builds after they leave.

DRAFT

Vignette — The V Corps G3 OSDK Application Review

A TM-40F engineer submits a PR for a new OSDK application serving the V Corps G3 — a near-real-time force disposition display used during exercises. The application is functionally correct: it queries the right Object Types, displays the right data, and handles the primary use case correctly.

The TM-50L lead conducting the review identifies the following issues, none of which were caught in functional testing: - The application loads all force disposition Objects at startup (4,200 at current exercise scale) rather than filtering to the G3's AOR on load. This will cause long initial load times as exercise scale increases. - There is no cache invalidation logic — the application caches data at load and never refreshes during a session. During a 12-hour exercise, the displayed data will be up to 12 hours stale. - Actions available to the G3 are also visible (though non-functional) to read-only roles due to UI logic that checks permissions server-side but renders the button before the check resolves. - There is no runbook for the application — no documentation of the subscription TTL, how to force a refresh, or what to do if the Ontology subscription drops.

The TM-50L lead does not reject the PR. They document each finding with the reasoning for why it matters operationally, assign severity (blocking vs. advisory), and provide specific remediation guidance. The conversation that follows the review raises the TM-40F engineer's understanding of OSDK caching, authorization patterns, and production documentation requirements — knowledge they will carry into the next five applications they build.

DRAFT

SUMMARY TABLE — TM-50L CONCEPTS AT A GLANCE

Section	Core Concept	Key Discipline
1. Platform Engineer	Engineering leadership without the title	Raise the floor, not just the ceiling
2. Platform Architecture	Spark execution model; symptoms-to-root-cause	Diagnose from evidence; fix the design, not the resources
3. Ontology at Scale	Design patterns for scalability	Evaluate before implementing; fan-out and sparsity as risk signals
4. OSDK Architecture	Caching, subscriptions, auth patterns	Access-pattern-first design; authorization defense in depth
5. Security Engineering	Security as design constraint	Trace every data flow; audit trail for every Action
6. CI/CD	Governance without bottlenecks	Automate verification; reserve humans for approval
7. Code Review	Four-dimensional review; developer mentorship	Blast radius awareness; feedback that teaches
8. Documentation	Documentation as operational infrastructure	The 0300 test; co-location; review cycles
9. Failure Modes	Over-engineering, observability debt, security as compliance	Design discipline; team development as mission requirement

This guide is a conceptual companion to TM-50L. It does not replace task-based training. Qualification requires demonstrated performance of TM-50L tasks under the conditions and to the standards specified in TM-50L.

USAREUR-AF Operational Data Team — MSS Platform Engineering Version 1.0 — 2026

DRAFT